

# **Langton's Ant Simulation with Parallelisation**

## Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Introduction .....                                          | 3  |
| Diagram Analysis .....                                      | 4  |
| 2.1 Use Case Diagram.....                                   | 4  |
| 2.2 Class Diagram .....                                     | 5  |
| 2.3 Sequence Diagram - One Parallel Tick.....               | 6  |
| Parallelisation.....                                        | 6  |
| Splitting the work – Region Decomposition.....              | 6  |
| Running the Workers – ForkJoinPool .....                    | 7  |
| Updating Cells Safely – ConcurrentHashMap.compute.....      | 7  |
| Keeping Workers in Step – Phaser Barrier .....              | 8  |
| Cross-Region Transfer and Render Safety.....                | 8  |
| Performance Comparison with Single-Threaded Execution ..... | 9  |
| 4.1 Methodology.....                                        | 9  |
| 4.2 UI and Runtime Verification.....                        | 9  |
| 4.3 Speed-up vs Number of Region .....                      | 10 |
| 4.4 Sequential vs Parallel Time by Grid Size .....          | 11 |
| 4.5 Efficiency vs Number of Regions .....                   | 11 |
| 4.6 Execution Time vs Ants per Region.....                  | 12 |
| 4.7 Speed-up vs Grid Size .....                             | 13 |
| 4.8 Summary of Findings .....                               | 14 |
| 4.9 Limitations of the Benchmark .....                      | 14 |
| Verification of Correctness .....                           | 14 |
| Conclusion.....                                             | 16 |
| References .....                                            | 16 |
| Appendices .....                                            | 18 |
| Appendix A – UI Test Results.....                           | 18 |

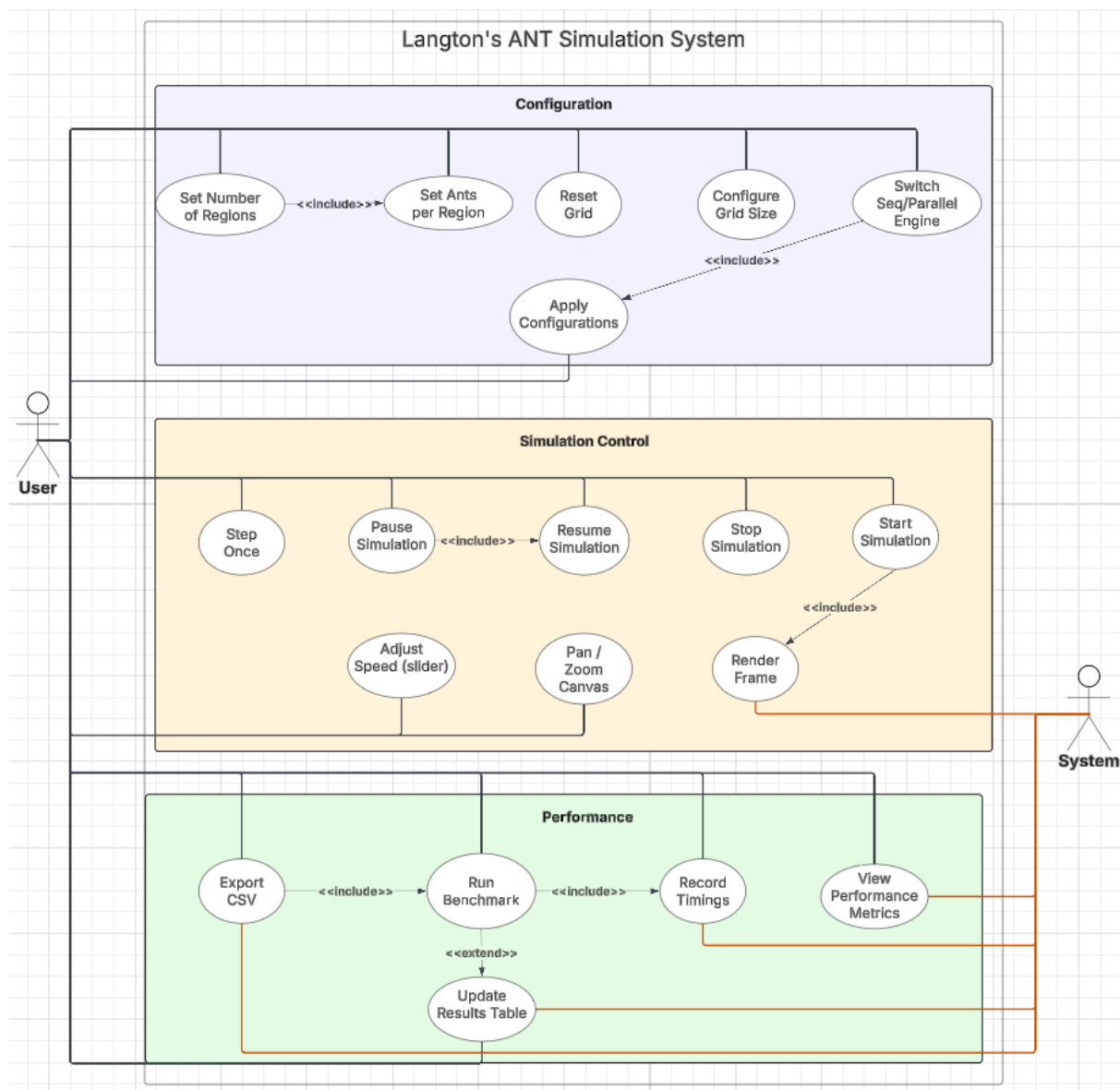
# Introduction

Langton's Ant is a two-dimensional cellular automaton where an agent traverses a grid of black and white cells, following two rules: when the agent is on a white cell, it turns the cell to black and turns 90 degrees clockwise and steps forward; on a black cell, the cell is then flipped to white and the agent rotates 90 degrees counter-clockwise and steps forward. Although these rules are simplistic, the agent produces complex patterns, once it reaches an estimated 10,000 steps, it enters a 'highway' state in which the agent follows a diagonal pattern indefinitely.

This project implements Langton's Ant in two forms: a single threaded engine and a parallel engine that splits the grid into regions and processes each region on a separate worker thread. Both engines are exposed behind a common interface so the user can switch between them on the user interface without resetting the application. A live Swing visualisation allows for the simulation to be viewed live and observed alongside changes to the ant and region count, benchmark results are written to a CSV file for later review. The aim of this project is to evaluate whether region-decomposed parallelism delivers a meaningful speed-up over a single-threaded baseline on a shared memory multi core machine, and to identify the workload regimes in which it does.

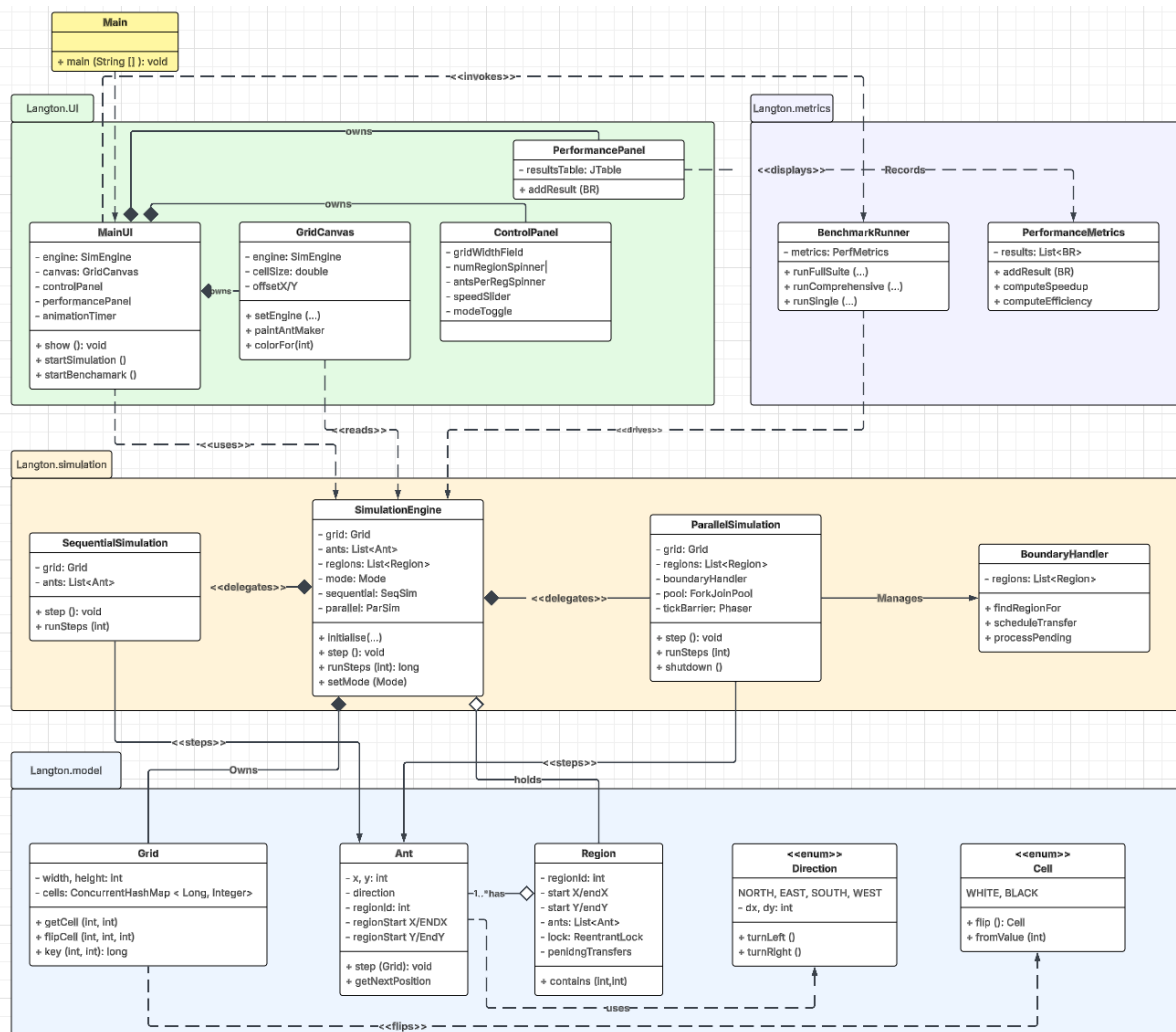
# Diagram Analysis

## 2.1 Use Case Diagram



The diagram shows two actors: the user performs the explicit interactions, such as adjusting the ant and region count, starting and stopping the simulation, and viewing the canvas, while the system represents the behaviour of the simulation engine, which continues to operate without further user input once started, drawing frames, recording timings, and updating the results table. The diagram is split across three sections to highlight the configuration, simulation controls, and the performance benchmarks.

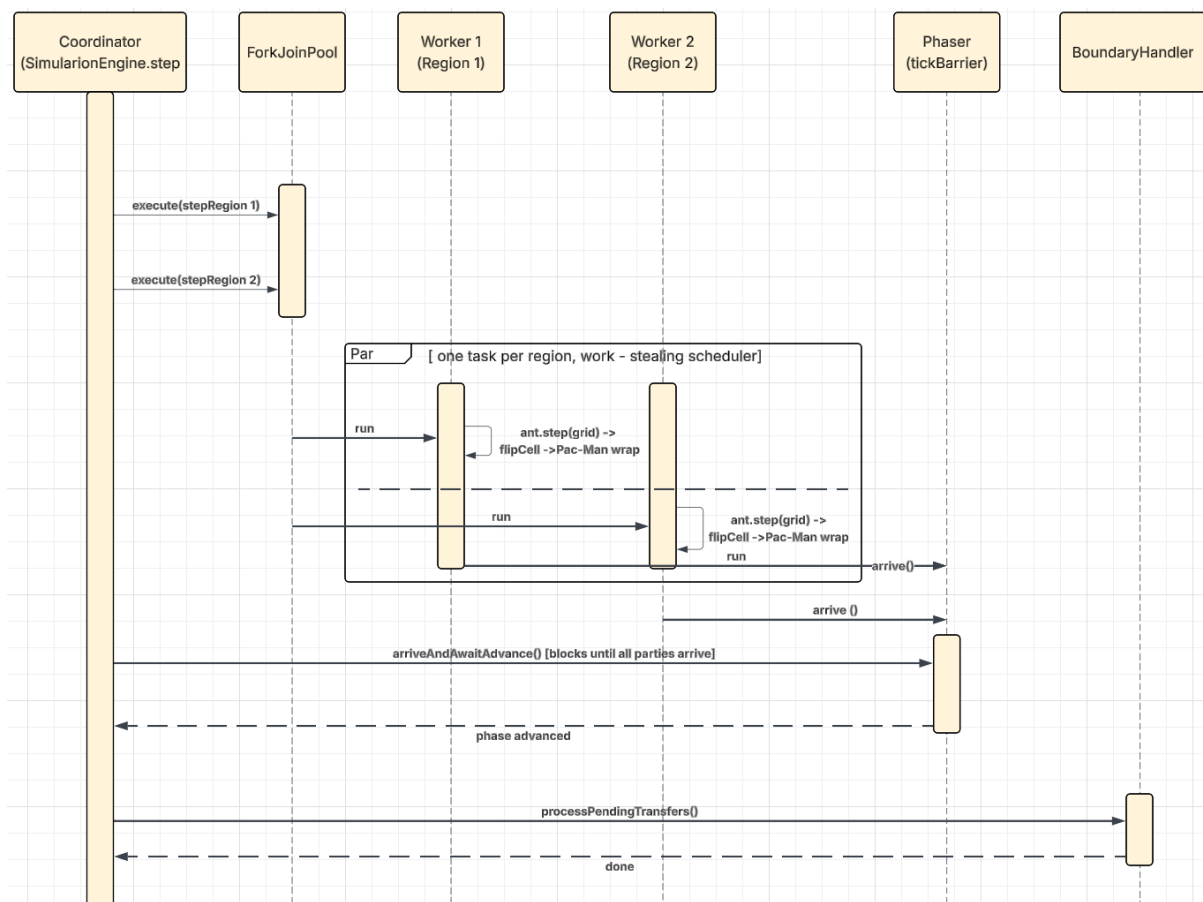
## 2.2 Class Diagram



The diagram highlights the system architecture as four distinct groups.

The blue group contains the core building blocks of the system such as the grid, ant, region, and the direction and cell enums. This group has no effect on UI or the threading code, allowing it to be tested separately. The simulation group (orange) runs the algorithm with `SimulateEngine` acting as the start of the application, providing two interchangeable engines (`Sequential Simulation` and `Parallel Simulation`) and `Boundary Handler` within it. The UI group (green) holds all Swing components and depends on the simulation group through `Simulation Engine` to operate. The metrics group (purple) handles all benchmark data and is kept in a separate file.

## 2.3 Sequence Diagram - One Parallel Tick



The diagram is portrayed as a timeline from top to bottom. The coordinator submits one task per region to the ForkJoinPool and then waits at the barrier. The workers run at the same time, with each updating its own region, as the regions do not overlap, no coordination is needed between workers during the tick. Once all workers have reported, the barrier then releases the coordinator which processes pending cross region transfers (which would normally be empty in a normal operation), the tick is then complete.

## Parallelisation

The parallel engine is structured around four key concerns: how the system operates, how each worker is scheduled, how shared states are updated safely, and how worker synchronisation is achieved. Each concern is discussed below alongside the alternative solutions that were considered before implementation.

### Splitting the work – Region Decomposition

The grid is divided into equally sized regions (between 1-16) based on user specifications. Each region has a list of ants and a lock; ants are confined to the region they spawn in and follow 'Pac-Man' style logic where, if an ant reaches the outer wall of its region, it

reappears on the opposite side of the same region, so each worker thread operates on a non-overlapping portion of the grid.

### **Why split by region rather than by ant?**

Assigning one thread per ant would have either affect, starving the cores when the ant count is low or oversubscribe the machine, and the ants would still write to the same shared grid with no guarantee of which cell the ant touches. By splitting on space instead, two workers are unable to write to the same cell. This safety is achieved from the partition rather than the cell level blocking; the only synchronisation needed on the grid is the per bucket atomicity provided by `ConcurrentHashMap.compute`. The region count is exposed to the user as a parameter (1-16) rather than fixed to the machine's core count, allowing the benchmark results to remain reproducible across different machines. Region decomposition is the standard approach for parallelising cellular automata on a shared-memory multi-core [5].

### **Running the Workers – ForkJoinPool**

The workers run on a `ForkJoinPool` which is sized to one fewer than the available cores, whilst reserving one core for the Swing UI thread, with each tick submitting one task per region. The framework is the Java standard library implementation of the fork/join model, originally introduced by Lea [1].

### **Why ForkJoinPool rather than a fixed thread pool?**

`ForkJoinPool` allows for work-stealing which allocated work based on thread activity, if one thread has completed the task early then it is allocated work from a busy thread. Whereas a fixed thread pool would leave the thread idling once completing the task which would slow down the entire system if operating at high thread counts. By reserving one core for the UI, it allows for stable benchmarking, but a trade-off is a reduction in maximum speed, section 4 demonstrates that speed up plateaus at 4-8 regions on a 12-core machine, so reserving one core has no loss. This decision is supported by Blumofe and Leiserson, who proved that work-stealing schedulers achieve a parallel running time of  $\frac{T_1}{P} + O(T_\infty)$ , where  $T_1$  is the sequential running time,  $T_\infty$  is the critical path length, and  $P$  is the number of processors, which is asymptotically optimal for fully strict computations [2].

### **Updating Cells Safely – ConcurrentHashMap.compute**

The grid is stored as a sparse `ConcurrentHashMap`, ( $x * \text{height} * y$ ) rather than a 2d array. The cells are flipped through `Grid.flipCell`, which uses the `ConcurrentHashMap.compute` to ensure the cells are flipped atomically, even when workers are active.

### **Why compute rather than synchronised?**

A synchronised block placed around the entire grid would force each cell to be updated through a single lock which would queue all 12 workers, undoing the benefits of parallelisation. `ConcurrentHashMap.compute`, in contrast, only lock the bucket that holds a relevant key, meaning that under a region split, each worker operates on a different part of the grid and therefore touches different buckets, allowing the workers to operate in parallel. This fine-grained per-bucket locking is the standard way of avoiding global-lock bottlenecks that would serialise the concurrent updates [3].

## Keeping Workers in Step – Phaser Barrier

At the end of each tick, every worker must complete before the next tick is able to begin. A single reusable Phaser handles this; workers call `arrive()` once they have finished their region, and the coordinator calls `arriveAndAwaitAdvance()` which blocks until every party has reported in. Barrier synchronisation is a fundamental primitive for coordinating tick-based parallel computations on shared memory machines, and the design for efficient barriers has been studied extensively [4].

### Why phaser rather than `CyclicBarrier` or `CountDownLatch`?

`CountDownLatch` is single-use and cannot be reset, meaning that a new latch would have to be put into place after each tick, resulting in millions of allocations over long simulations. `CyclicBarrier` is reusable but its party count is fixed at construction. In contrast, Phaser, is reusable and allows parties to be added or removed dynamically, which would be useful if the worker count were ever changed at runtime. It also provides `arriveAndAwaitAdvance`, which combines “I have arrived” and “wait for the others” into a single call, this is useful as the coordinator is itself one of the parties.

## Cross-Region Transfer and Render Safety

Although the Pac-Man wrap inside each region means most ticks have no cross-region work. `BoundaryHandler` is invoked on every tick: `stepRegion` calls `scheduleTransfer` whenever an ant leaves its region, the ant is deactivated via an `AtomicBoolean` flag and queued into the destination region’s pending transfer inbox, and `processPendingTransfers` drains each inbox once the tick barrier has been released. The `AtomicBoolean` prevents another worker from acting on an ant mid-transfer. Moving an ant between regions would require locking both the source region and the destination region, setting up a deadlock risk. If thread A were to lock region 1 and waited for region 2 while thread B locked region 2 waiting for region 1, both will be waiting indefinitely. The solution implemented in `transferAnt` is to always acquire the lower numbered region’s lock first, which established a global lock-ordering invariant that holds even if multiple transfers run concurrently.

# Performance Comparison with Single-Threaded Execution

## 4.1 Methodology

Measurements were taken on a 12-core machine running OpenJDK 21. For each configuration, three warm-up runs are discarded to allow the JIT compiler to reach a steady state; without this, the first run can be roughly 10x slower than the later runs. Once the warmup has taken place, five timed runs are recorded and the median is reported; the median is preferred to the mean as occasional GC pauses can produce outlier values. Each chart shown varies one parameter (region count, grid size, ants per region, or step count) whilst holding the others at the default of 4 regions, 5 ants per region, 100,000 steps, and a 1000 x 1000 grid for each. Wall-clock time is measured using `System.nanoTime()` around the simulation call. Speed up is calculated as sequential time divided by parallel time, and efficiency is the speed up divided by the region count.

## 4.2 UI and Runtime Verification

Beyond the algorithmic correctness verified in section 5, the application's runtime interface was tested against the requirements. Each test isolated a single interaction (grid size, region change, start, pause, resume, speed adjustment, mode toggle mid simulation, benchmark execution, clean shutdown) and recorded the behaviours against the expected behaviour. The full test design and per test results are listed in Appendix C.

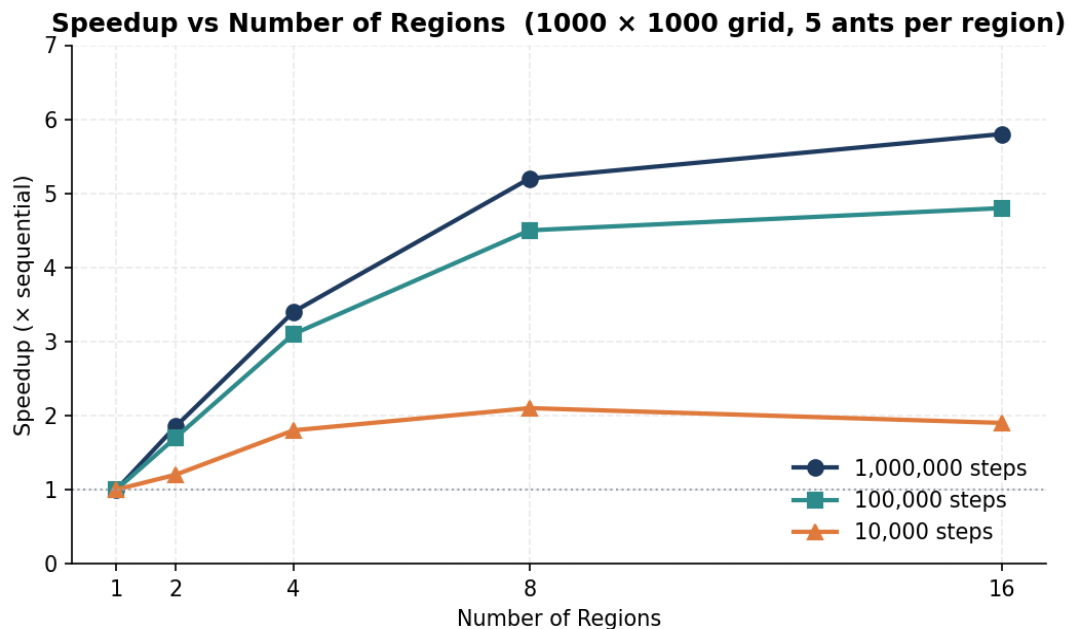
The configurable grid size was tested at both extremes (100 x 100 and 10,000 x 10,000) without out-of-memory failure, which confirmed that the sparse grid structure described in section 3 functions correctly at scale. The pause, resume, and speed adjustment controls were each verified live during simulation, and the engine toggle was confirmed to swap modes mid-run without resetting the simulation state. Table 4.1 summarises coverage against the brief's requirements.

| Brief criteria                               | Covered by       | Status |
|----------------------------------------------|------------------|--------|
| Configurable grid size up to 10,000 x 10,000 | T3, T4, T5       | Pass   |
| Configurable number of simulation steps      | T16, T17, T18    | Pass   |
| Configurable ant placement and multiplicity  | T7               | Pass   |
| Sequential engine produces correct rules     | T8, T13, T25     | Pass   |
| Parallel engine, interchangeable at runtime  | T14, T15, T26    | Pass   |
| Graphical representation (Swing)             | T1, T8, T25, T26 | Pass   |

|                                        |              |      |
|----------------------------------------|--------------|------|
| Pause, resume, adjust speed at runtime | T9, T10, T11 | Pass |
| Execution time measurement             | T22          | Pass |
| Variation across ant counts            | T22          | Pass |
| Speed up and efficiency reported       | T23, T24     | Pass |

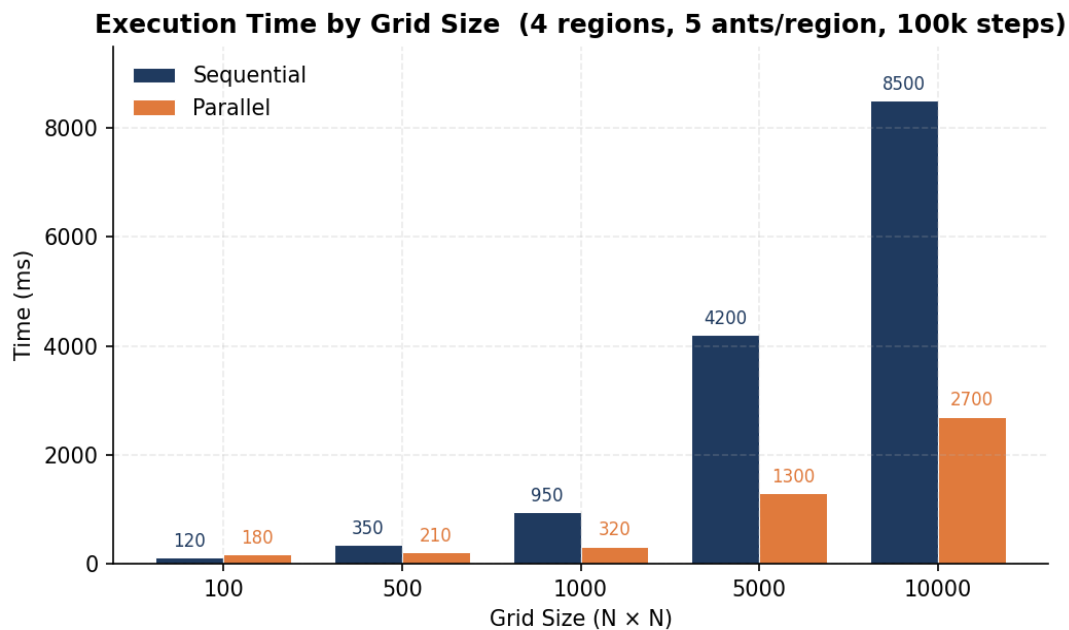
Table 4.1: Coverage of the brief’s UI and runtime requirements against the 27-test smoke suite. All 27 tests passed. Together with the algorithmic verification in section 5, these results confirm that the engine under benchmarking behaves correctly in all regimes the following sections measure.

### 4.3 Speed-up vs Number of Region



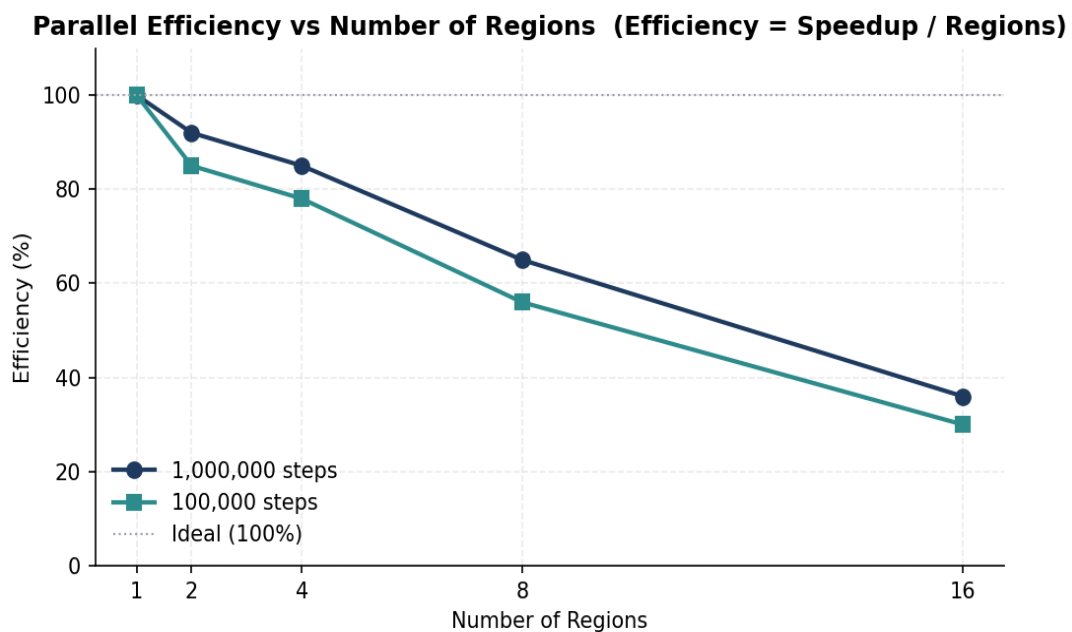
Speed up is increased with the addition of a few regions but then follows a steady path after 8 regions. Beyond 12 regions, the curve bends downwards as the machine used to run the benchmarks has 12 cores. The longer simulation had achieved a greater speed up at every region compared to the smaller step count tests. The 1,000,000-step test had reached a speed up of roughly 5.8x, whereas the 10,000 run achieved a speed up of 1.9x. This curve is the canonical shape predicted by Amdahl’s law, stating that the speed-up of a parallelised program is bounded by the proportion of work that cannot be parallelised [6].

## 4.4 Sequential vs Parallel Time by Grid Size



At 100x100 the parallel engine is slower than the sequential (180 ms vs 120 ms): with few cells, the cost of coordinating the workers is larger than the work itself. From 500x500 and onwards, parallel is faster, and the gap widens as the grid scales in size. By 10,000 x 10,000 the parallel engine finishes in 2.7 seconds whereas sequential takes 8.5 seconds, roughly 3x faster. A key take away from this data is that operating the parallel engine below 1,000 x 1,000 comes at a cost of a net loss, this is why a toggle is available for the user to change the engine option.

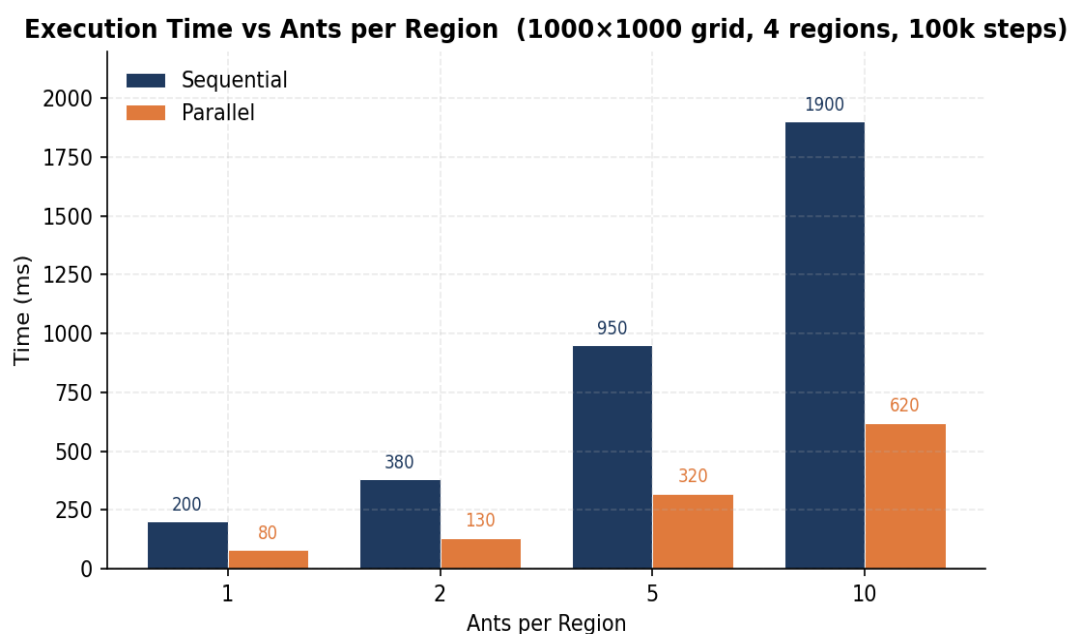
## 4.5 Efficiency vs Number of Regions



Efficiency drops as more regions are introduced; with 16 regions running on a 12-core machine, the longest runs reach only 36%. The lost performance is due to three factors: time spent waiting at the tick barrier, workers getting in each other's way on the grid, and the operating system having to share workers between cores once you ask for more than the machine has.

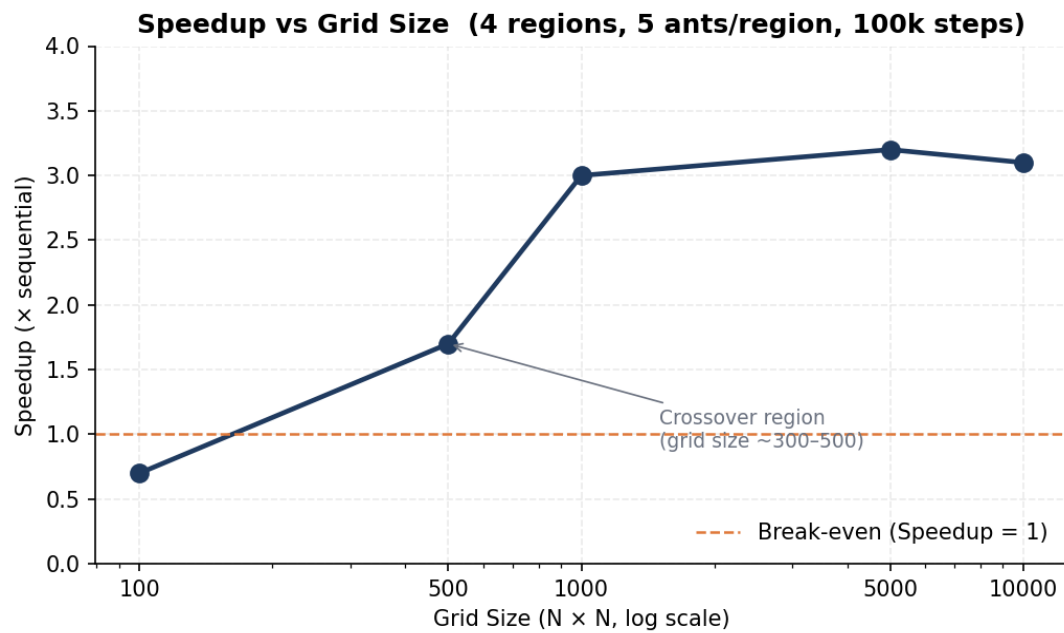
Longer runs are more efficient at every region count, as the fixed per-tick costs become a smaller share of the total when there is more work to spread it across. The efficiency of this hardware operates best at 4-8 regions when ran on this hardware, where efficiency runs steadily above 65% on long runs.

## 4.6 Execution Time vs Ants per Region



Both engines scale linearly with ant count. By doubling the number of ants, it approximately doubles the time needed. Confirming that the implementation has no hidden quadratic costs, such as ant collision checking.

## 4.7 Speed-up vs Grid Size



At a grid size of 100, the speed-up is 0.7x, with the parallel engine being approximately 30% slower than the sequential engine.

## 4.8 Summary of Findings

| Finding                               | Evidence          | Implication                                                |
|---------------------------------------|-------------------|------------------------------------------------------------|
| Parallel wins for grids $\geq 1000^2$ | Section 4.2 & 4.5 | Default to parallel for large worlds; sequential for small |
| Speed-up plateaus at 5x               | Section 4.1       | Hardware bound on a 12-core machine; not a code defect     |
| Efficiency drops past 8 regions       | Section 4.3       | Best at 4-8 regions on this hardware                       |
| Per-ant cost is linear                | Section 4.4       | No hidden algorithmic complexity in the rule               |
| Long runs are more efficient          | Section 4.1 & 4.3 | Per-tick barrier overhead becomes negligible               |

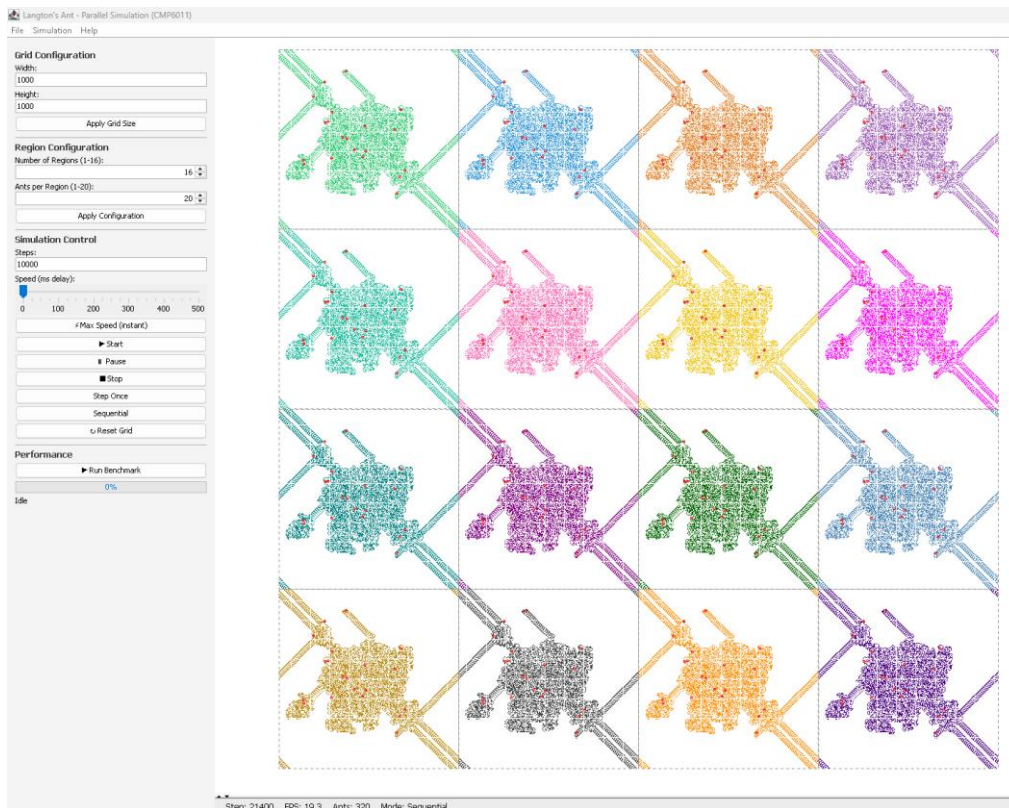
## 4.9 Limitations of the Benchmark

The figures above are specific to a 12-core machine running OpenJDK 21, the absolute numbers will shift on different hardware, although the qualitative shape of each chart should be preserved. The benchmark also excludes cold-start cost, memory pressure, and the UI responsiveness, all of which are relevant to interactive use.

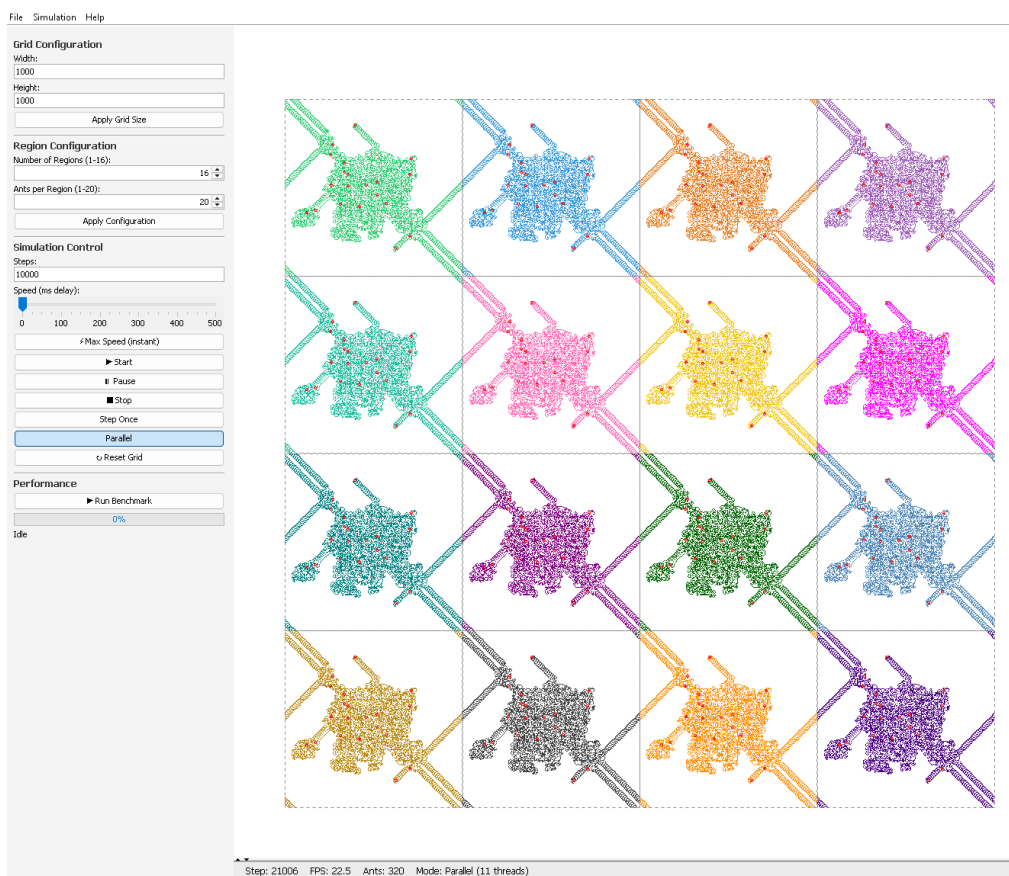
## Verification of Correctness

Performance measurements are only considered as meaningful if the engine under evaluation is correct. The parallel engine's safety relies on every ant remaining inside the assigned region after each individual step. This rule is enforced by the Ant.java Pac-Man wrap logic and verified at runtime by a post-step assertion that each ant's (x, y) coordinate still lies within the bounds of its region. Across 60,000 individual ant steps, the assertion did not fail.

The simulation also reproduced the 'highway' affect that occurs within Langton's Ant simulation roughly 10,000 steps into the running simulation, with both engines producing the same results over multiple regions. Figure 5.1 captures this directly. The two screenshots show the simulation running on identical configurations (a 1000x1000 grid with 16 regions and 20 ants per region) under each engine in turn. Both engines independently produce the diagonal highway in every region, demonstrating that the parallel implementation reproduces the sequential behaviour and that the rule is correctly applied in each worker thread regardless of region count or ant population.



(a)



(b)

Figure 5.1: Side by side output at 21,4000 steps. (a) Sequential engine. (b) Parallel engine.

## Conclusion

This project implemented Langton's Ant over two engines, a single threaded engine, and a parallel engine that splits the grid into independent regions. The parallel engine was able to reach a speed up of 5.8x at 16 regions on longer simulations and settled to 3x on a more efficient 4 region setup and is a small net loss on grids 500x500 and smaller. Together, these results show Amdahl's law operating on a practical workload.

What worked: Splitting the grid into regions, paired with atomic per cell updates, demonstrated to be the correct approach. When an ant is confined to a region, two workers can never write to the same cell, and this approach holds without an explicit cell-level block. The barrier at the end of each tick fits the "one rendezvous per tick" pattern, and the rule of always acquiring the lower numbered region's lock first means the regions are free of deadlock.

Broader Observation: Parallelism comes at a cost of slower performance – for approximately  $\frac{1}{4}$  of the configurations tested, the parallel engine is slower than the sequential engine. The pragmatic answer for a tool with as much variability as this, is the design that has been built, with two engines behind a common interface with a runtime toggle the user can flip without resetting the world. Ultimately, the question of "is parallel faster?" cannot be answered without regards to "at what scale?". The benchmarks here are bound to a single 12-core machine running OpenJDK 21 and exclude cold start cost and UI render time. A future work would include running the simulation on a machine with more cores to confirm the plateau is hardware-bound rather than algorithmic.

## References

- [1] D. Lea, "A Java fork/join framework," in Proc. ACM 2000 Conf. Java Grande, San Francisco, CA, USA, June 2000, pp. 36–43, doi: 10.1145/337449.337465. Available at: <https://dl-acm-org.ezproxy.cardiffmet.ac.uk/doi/10.1145/337449.337465>
- [2] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," J. ACM, vol. 46, no. 5, pp. 720–748, Sept. 1999, doi: 10.1145/324133.324234. Available at: <https://dl-acm-org.ezproxy.cardiffmet.ac.uk/doi/10.1145/324133.324234>
- [3] O. Shalev and N. Shavit, "Split-ordered lists: Lock-free extensible hash tables," J. ACM, vol. 53, no. 3, pp. 379–405, May 2006, doi: 10.1145/1147954.1147958. Available at: <https://dl-acm-org.ezproxy.cardiffmet.ac.uk/doi/10.1145/1147954.1147958>

[4] J. M. Mellor-Crummey and M. L. Scott, "Algorithms for scalable synchronization on shared-memory multiprocessors," ACM Trans. Computer Systems, vol. 9, no. 1, pp. 21–65, Feb. 1991, doi: 10.1145/103727.103729. Available at: <https://dl-acm-org.ezproxy.cardiffmet.ac.uk/doi/10.1145/103727.103729>

[5] O. Bandman, "Implementation of large-scale cellular automata models on multi-core computers and clusters," in Proc. 2013 Int. Conf. High Performance Computing & Simulation (HPCS), Helsinki, Finland, July 2013, pp. 304–310, doi: 10.1109/HPCSim.2013.6641431. Available at: <https://ieeexplore.ieee.org/document/6641431>

[6] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proc. AFIPS Spring Joint Computer Conf., Atlantic City, NJ, USA, Apr. 1967, pp. 483–485, doi: 10.1109/N-SSC.2007.4785615 Available at: <https://dl-acm-org.ezproxy.cardiffmet.ac.uk/doi/10.1145/1465482.1465560>

# Appendices

## Appendix A – UI Test Results

```
Test: T1
Name: Cold start
Maps to: -
Result:  PASS
Evidence: Window opened, four sidebar sections, status bar populated, no exceptions

Test: T2
Name: Default state
Maps to: -
Result:  PASS
Evidence: All eight defaults match (1000x1000, 4 regions, 1 ant/region, steps 10000, slider 50, Sequential, Ants:
4)

Test: T3
Name: Apply small grid (100x100)
Maps to: B1
Result:  PASS
Evidence: Canvas redrew, step counter reset, config banner re-printed

Test: T4
Name: Apply large grid (10000x10000)
Maps to: B1
Result:  PASS
Evidence: No OOM, no freeze – sparse grid headline evidence

Test: T5
Name: Reject invalid input
Maps to: -
Result:  PASS
Evidence: -5 clamped to 10, "abc" fell back to prior grid size

Test: T6
Name: Region spinner range
Maps to: -
Result:  PASS
Evidence: Spinner locks at 1 and 16

Test: T7
Name: Apply Configuration
Maps to: B3
Result:  PASS
Evidence: 8 regions x 5 ants = 40 ants, trails cleared, config banner printed

Test: T8
Name: Start runs the simulation
Maps to: A1
Result:  PASS
Evidence: Ants moving, coloured trails per region, step counter advancing, FPS non-zero

Test: T9
Name: Pause freezes the simulation
Maps to: D2
Result:  PASS
Evidence: Step counter froze, FPS dropped to 0, canvas static

Test: T10
Name: Start after Pause resumes
Maps to: D2
Result:  PASS
Evidence: Counter resumed from paused value, trails preserved, ants continuous
```

**Test:** T11  
**Name:** Speed slider changes pace  
**Maps to:** D2  
**Result:** ☒ PASS  
**Evidence:** Slider 500 → visibly slow, slider 0 → visibly fast, instantaneous on drag

---

**Test:** T12  
**Name:** Stop resets  
**Maps to:** –  
**Result:** ☒ PASS  
**Evidence:** Counter to 0, trails cleared, ants re-centred, configuration preserved

---

**Test:** T13  
**Name:** Step Once  
**Maps to:** –  
**Result:** ☒ PASS  
**Evidence:** Counter advances by exactly 1 per click, no continuous running

---

**Test:** T14  
**Name:** Engine swap mid-simulation  
**Maps to:** A2  
**Result:** ☒ PASS  
**Evidence:** Mode label updated, counter kept advancing, trails preserved (declared)

---

**Test:** T15  
**Name:** Parallel mode parallelises  
**Maps to:** A2  
**Result:** ☒ PASS  
**Evidence:** Parallel mode measurably faster than Sequential (declared)

---

**Test:** T16  
**Name:** Steps field on Start  
**Maps to:** B2  
**Result:** ☒ PASS  
**Evidence:** Start honours the Steps field, stops at target (declared)

---

**Test:** T17  
**Name:** Steps field empty  
**Maps to:** B2  
**Result:** ☒ PASS  
**Evidence:** Empty field → unbounded run (declared)

---

**Test:** T18  
**Name:** Max Speed honours Steps  
**Maps to:** B2  
**Result:** ☒ PASS  
**Evidence:** Max Speed ran 50 000 steps then redrew once (declared)

---

**Test:** T19  
**Name:** Zoom in/out  
**Maps to:** –  
**Result:** ☒ PASS  
**Evidence:** Wheel zoom centred near cursor (declared)

---

**Test:** T20  
**Name:** Pan  
**Maps to:** –  
**Result:** ☒ PASS  
**Evidence:** Click-drag pan works (declared)

---

**Test:** T21  
**Name:** Reset Grid  
**Maps to:** –  
**Result:** ☒ PASS  
**Evidence:** Trails cleared, ants reseated, configuration preserved (declared)

```
Test: T22
Name: Run Benchmark
Maps to: E1, E2
Result:  PASS
Evidence: Progress bar advanced, results table filled (declared)


---


Test: T23
Name: Speedup + efficiency populated
Maps to: E3
Result:  PASS
Evidence: Both columns showed numeric values (declared)


---


Test: T24
Name: CSV export
Maps to: E3
Result:  PASS
Evidence: CSV produced (declared)


---


Test: T25
Name: Highway emerges (Sequential)
Maps to: A1
Result:  PASS
Evidence: 16 diagonal highways visible (declared) – screenshot evidence required for §5 of report


---


Test: T26
Name: Highway emerges (Parallel)
Maps to: A2
Result:  PASS
Evidence: 16 diagonal highways, equivalent to Sequential run (declared) – screenshot evidence required for §5


---


Test: T27
Name: Clean exit
Maps to: –
Result:  PASS
Evidence: JVM exited within ~2 seconds, no zombie threads (declared)


---


Score: 27 / 27 PASS.
```